

Projet numérique - contrôle stochastique

Reproduction critique d'algorithmes de réseaux de neurones pour des problèmes de contrôle stochastique en horizon fini

D'après Bachouch, Huré, Langrené et Pham

Yvann Barclais Abdellah Rebaine

Master 2 Probabilité & Finance - Sorbonne Université

Année 2025–2026

Énoncé du sujet choisi

Sujet. Reproduire et analyser numériquement l'article de Bachouch, Huré, Langrené et Pham, *Deep neural networks algorithms for stochastic control problems on finite horizon : numerical applications*.

Table des matières

Énoncé du sujet choisi	1
1 Introduction	3
2 Cadre mathématique général	3
2.1 Problème de contrôle stochastique discret	3
2.2 Programmation dynamique, fonction valeur et politique optimale	3
2.3 Approximation par réseaux de neurones	3
2.4 Spécificités des réseaux de neurones utilisés dans le code	4
2.5 Approximation Monte Carlo des pertes	4
3 Algorithmes étudiés	4
3.1 NNContPI : apprentissage direct par performance iteration	4
3.2 ClassifPI : cas des contrôles discrets	5
3.3 Hybrid-Now : régression now de la politique et de la valeur	5
3.4 Hybrid-LaterQ : regress later et quantification	6
3.5 Qknn : quantification et plus proches voisins	6
3.6 ClassifHybrid : contrôle mixte du micro-réseau	6
3.7 Comparaison critique des méthodes	7
4 Méthodologie expérimentale	7
4.1 Modes d'exécution et conséquences	7
4.2 Intervalles de confiance	7
5 Expérience 1 - PDE semi-linéaire	7
5.1 Modèle continu et discret	7
5.2 Notre modèle de reproduction	8
5.3 Test grande dimension : erreur relative	8
5.4 Trajectoires en dimension 2	9
5.5 Balayage en γ	9
5.6 Comparaison critique des approches utilisées	10
6 Expérience 2 - Contrôle linéaire-quadratique	10
6.1 Modèle et solution de référence	10
6.2 Notre modèle de reproduction	10
6.3 Résultats graphiques et numériques	11
6.4 Comparaison critique des approches utilisées	12
7 Expérience 3 - Couverture quadratique d'option	12
7.1 Modèle	12
7.2 Notre modèle de reproduction	13
7.3 Résultats graphiques et numériques	13
7.4 Comparaison critique des approches utilisées	14
8 Expérience 4 - Stockage de gaz	14
8.1 Modèle	14
8.2 Notre modèle de reproduction	14
8.3 Comparaison table à table	15
8.4 Résultats graphiques	15
8.5 Comparaison critique des approches utilisées	17

9	Expérience 5 - Gestion d'un micro-réseau	17
9.1	Modèle	17
9.2	Notre modèle de reproduction	17
9.3	Comparaison numérique	17
9.4	Résultats graphiques	18
9.5	Comparaison critique des approches utilisées	20
10	Bilan critique et robustesse	20
10.1	Reproduction qualitative vs quantitative	20
10.2	Sources d'écart	20
10.3	Comparaison finale des méthodes	20
11	Implémentation informatique	21
11.1	Paramètres neuraux dans les scripts	21
11.2	Dépendances	21
11.3	Commandes utiles	21
12	Conclusion	21

1 Introduction

Les problèmes de contrôle stochastique en horizon fini consistent à choisir, à chaque date, une action adaptée à l'information disponible afin de minimiser un coût espéré ou de maximiser un gain espéré. Ils apparaissent notamment en finance, en gestion de l'énergie et dans l'approximation numérique d'équations aux dérivées partielles non linéaires.

Le cadre théorique repose sur le principe de programmation dynamique, qui introduit la fonction valeur et caractérise les politiques optimales par récurrence arrière. En pratique, cette méthode devient coûteuse en grande dimension, car les discrétisations classiques de l'espace d'état souffrent de la malédiction de la dimension. L'article étudié propose donc d'utiliser des réseaux de neurones profonds pour approximer les contrôles optimaux et, selon les méthodes, les fonctions valeur.

Les algorithmes considérés dans ce rapport sont NNCONTPI, CLASSIFPI, HYBRID-NOW, HYBRID-LATERQ, QKNN et CLASSIFHYBRID. Ils sont comparés à des références analytiques ou numériques lorsque celles-ci sont disponibles : formule de Hopf–Cole, solution de Riccati, formule de Bertsimas–Kogan–Lo, ainsi que des stratégies naïves comme le contrôle nul.

L'objectif du rapport est de reproduire plusieurs expériences numériques du papier, puis de comparer nos résultats aux figures et valeurs publiées. On distingue une reproduction qualitative, lorsque les tendances graphiques et les structures de décision sont similaires, d'une reproduction quantitative, lorsque les valeurs numériques sont proches. Cette distinction est importante car certains écarts peuvent venir du budget Monte Carlo, du nombre d'epochs, du choix des réseaux ou de simplifications dans le code.

2 Cadre mathématique général

2.1 Problème de contrôle stochastique discret

On considère un horizon fini N et un processus d'état $X = (X_n)_{0 \leq n \leq N}$ à valeurs dans $\mathbb{R}^d$. La dynamique contrôlée est donnée par

$$X_{n+1} = F(X_n, \alpha_n, \varepsilon_{n+1}), \quad n = 0, \dots, N-1, \quad X_0 = x_0, \quad (1)$$

où les bruits (ε_n) sont i.i.d. Le contrôle α_n est $\mathcal{F}_n$ -mesurable : il ne peut dépendre que de l'information disponible au temps n . Il prend ses valeurs dans un ensemble admissible $A \subset \mathbb{R}^q$.

Le coût associé à une stratégie $\alpha = (\alpha_0, \dots, \alpha_{N-1})$ est

$$J(\alpha) = \mathbb{E} \left[\sum_{n=0}^{N-1} f(X_n, \alpha_n) + g(X_N) \right], \quad (2)$$

où f est le coût courant et g le coût terminal. Le problème est

$$V_0(x_0) = \inf_{\alpha \in \mathcal{A}} J(\alpha). \quad (3)$$

Ce formalisme recouvre à la fois des problèmes naturellement discrets, par exemple la couverture d'option sur un arbre trinomial, et des discrétisations en temps de modèles continus, par exemple une EDS contrôlée.

2.2 Programmation dynamique, fonction valeur et politique optimale

La fonction valeur au temps n est

$$V_n(x) = \inf_{\alpha_n, \dots, \alpha_{N-1}} \mathbb{E} \left[\sum_{k=n}^{N-1} f(X_k, \alpha_k) + g(X_N) \mid X_n = x \right]. \quad (4)$$

À l'instant terminal, $V_N(x) = g(x)$. Pour $n = N-1, \dots, 0$, le principe de programmation dynamique donne

$$V_n(x) = \inf_{a \in A} Q_n(x, a), \quad Q_n(x, a) = f(x, a) + \mathbb{E} [V_{n+1}(F(x, a, \varepsilon_{n+1}))]. \quad (5)$$

Lorsque l'infimum est atteint, une politique optimale en feedback est

$$a_n^*(x) \in \arg \min_{a \in A} Q_n(x, a), \quad \alpha_n^* = a_n^*(X_n^*). \quad (6)$$

2.3 Approximation par réseaux de neurones

L'article remplace les objets inconnus par des familles paramétriques :

$$A(\cdot; \beta) : \mathbb{R}^d \rightarrow A, \quad \Phi(\cdot; \theta) : \mathbb{R}^d \rightarrow \mathbb{R}. \quad (7)$$

Le réseau A approxime la politique optimale, tandis que Φ approxime la fonction valeur. Si $A = \mathbb{R}^q$, la sortie peut être linéaire ; si A est borné, on ajoute une activation ou une projection ; si A est fini, on utilise une sortie *softmax*.

L'apprentissage ne se fait pas uniformément sur tout $\mathbb{R}^d$, mais sur des échantillons $X_n \sim \mu_n$, où μ_n est la mesure d'entraînement au temps n . Cette mesure est un choix numérique important. Si l'on sait que la trajectoire optimale reste dans une région D , on peut concentrer μ_n sur D . Sinon, on peut effectuer une phase d'exploration, simuler une politique approximative, puis réentraîner sur les lois empiriques des états visités.

2.4 Spécificités des réseaux de neurones utilisés dans le code

On n'utilise pas exactement la même pile logicielle que l'article : l'article historique est écrit avec TensorFlow, alors que notre implémentation est en PyTorch. Les réseaux sont néanmoins du même type : des perceptrons multicouches entièrement connectés. La classe centrale du code est MLP. Elle applique, sauf exception, la structure suivante :

$$x \longmapsto \text{Linear} \rightarrow \sigma \rightarrow \cdots \rightarrow \text{Linear},$$

où σ est l'activation ELU dans toutes les expériences principales. Le code permet aussi ReLU, *tanh* et GELU, mais les scripts de reproduction utilisent ELU, conformément à l'observation du papier selon laquelle ELU donne souvent de meilleurs résultats sur ces exemples. Les entrées sont normalisées par un normaliseur courant *RunningNormalizer*, ce qui stabilise la descente de gradient lorsque les composantes de l'état n'ont pas la même échelle.

Les sorties dépendent de la nature du contrôle :

- pour un contrôle continu non borné, la dernière couche est linéaire ;
- pour un contrôle continu borné, la sortie brute est transformée par une *tanh* puis projetée dans l'intervalle admissible ;
- pour un contrôle discret, la sortie est un vecteur de logits suivi d'une *softmax* ; lorsque le problème impose des contraintes d'état, les actions non admissibles sont masquées avant la *softmax* ou avant le choix glouton ;
- pour la couverture d'option, le code n'utilise pas un réseau totalement générique : il impose un ansatz affine pour la politique en richesse et quadratique pour la valeur.

L'optimisation utilise *Adam*. Les pertes incluent une régularisation L^2 , et certaines expériences utilisent un clipping du gradient. Le warm-start temporel est activé dans les configurations principales : le réseau au temps n est initialisé avec les poids du temps $n + 1$ lorsque c'est pertinent.

2.5 Approximation Monte Carlo des pertes

Les espérances dans les pertes d'apprentissage sont remplacées par des moyennes empiriques sur des mini-batches. Par exemple, la perte one-step de Hybrid-Now est approchée par

$$\frac{1}{M} \sum_{m=1}^M \left[f(X_n^{(m)}, A(X_n^{(m)}; \beta)) + \hat{V}_{n+1} \left(F(X_n^{(m)}, A(X_n^{(m)}; \beta), \varepsilon_{n+1}^{(m)}) \right) \right]. \quad (8)$$

La descente de gradient introduit donc une erreur statistique et une erreur d'optimisation. Ces deux erreurs sont visibles dans nos résultats : certains graphiques ont la bonne forme mais les valeurs finales restent éloignées du papier.

3 Algorithmes étudiés

3.1 NNContPI : apprentissage direct par performance iteration

NNContPI est adapté aux espaces de contrôle continus. Supposons que les politiques $\hat{a}_{n+1}, \dots, \hat{a}_{N-1}$ soient déjà apprises. À la date n , on cherche

$$\hat{\beta}_n \in \arg \min_{\beta} \mathbb{E} \left[f(X_n, A(X_n; \beta)) + \sum_{k=n+1}^{N-1} f(X_k^{\beta}, \hat{a}_k(X_k^{\beta})) + g(X_N^{\beta}) \right], \quad (9)$$

où $X_n \sim \mu_n$, puis

$$X_{n+1}^{\beta} = F(X_n, A(X_n; \beta), \varepsilon_{n+1}), \quad X_{k+1}^{\beta} = F(X_k^{\beta}, \hat{a}_k(X_k^{\beta}), \varepsilon_{k+1}).$$

Autrement dit, le réseau candidat n'est utilisé qu'au premier pas, puis on déroule les politiques futures déjà apprises.

Algorithm 1 NNContPI

-
- 1: **Entrée** : mesures d'entraînement $(\mu_n)_{0 \leq n < N}$
 - 2: **Sortie** : politiques approximées $(\hat{a}_n)_{0 \leq n < N}$
 - 3: **for** $n = N - 1, \dots, 0$ **do**
 - 4: Échantillonner $X_n \sim \mu_n$ et les bruits futurs.
 - 5: Simuler de n à N avec $A(\cdot; \beta)$ au premier pas.
 - 6: Utiliser ensuite les politiques futures $\hat{a}_{n+1}, \dots, \hat{a}_{N-1}$.
 - 7: Minimiser la perte (9) par descente de gradient.
 - 8: Poser $\hat{a}_n = A(\cdot; \hat{\beta}_n)$.
 - 9: **end for**
-

L'avantage est conceptuel : on apprend directement une stratégie. La limite est le coût. Quand n est petit, chaque évaluation de perte traverse presque tout l'horizon et tous les réseaux futurs. Dans nos expériences, cela explique que NNContPI soit plus lent que Hybrid-Now, même lorsqu'il donne une meilleure valeur sur le test LQ.

3.2 ClassifPI : cas des contrôles discrets

Lorsque $A = \{a_1, \dots, a_L\}$, l'action peut être choisie par classification. Le réseau produit

$$p(x; \beta) = (p_1(x; \beta), \dots, p_L(x; \beta)), \quad \sum_{\ell=1}^L p_\ell(x; \beta) = 1,$$

et la politique déterministe est

$$\hat{a}_n(x) = a_{\ell^*(x)}, \quad \ell^*(x) \in \arg \max_{\ell} p_\ell(x; \hat{\beta}_n). \quad (10)$$

La perte utilisée au temps n pondère le coût de chaque action par sa probabilité :

$$\hat{\beta}_n \in \arg \min_{\beta} \mathbb{E} \left[\sum_{\ell=1}^L p_\ell(X_n; \beta) \left(f(X_n, a_\ell) + \sum_{k=n+1}^{N-1} f(X_k^\ell, \hat{a}_k(X_k^\ell)) + g(X_N^\ell) \right) \right]. \quad (11)$$

Algorithm 2 ClassifPI

-
- 1: **Entrée** : actions $a_1, \dots, a_L$, mesures μ_n
 - 2: **for** $n = N - 1, \dots, 0$ **do**
 - 3: Évaluer, pour chaque action a_ℓ , le coût futur associé.
 - 4: Apprendre les probabilités $p_\ell(\cdot; \beta)$ en minimisant (11).
 - 5: Définir $\hat{a}_n(x)$ comme l'action de probabilité maximale.
 - 6: **end for**
-

Cette méthode est naturelle pour le stockage de gaz, où les actions sont injecter, attendre ou retirer. Sa faiblesse est qu'elle peut lisser des frontières de décision nettes.

3.3 Hybrid-Now : régression now de la politique et de la valeur

Hybrid-Now introduit une approximation explicite de la fonction valeur. On initialise $\hat{V}_N = g$. Au temps n , la politique est entraînée avec une perte one-step :

$$\hat{\beta}_n \in \arg \min_{\beta} \mathbb{E} \left[f(X_n, A(X_n; \beta)) + \hat{V}_{n+1}(F(X_n, A(X_n; \beta), \varepsilon_{n+1})) \right]. \quad (12)$$

Une fois $\hat{a}_n = A(\cdot; \hat{\beta}_n)$ fixée, on régresse la valeur :

$$\hat{\theta}_n \in \arg \min_{\theta} \mathbb{E} \left[\left(f(X_n, \hat{a}_n(X_n)) + \hat{V}_{n+1}(F(X_n, \hat{a}_n(X_n), \varepsilon_{n+1})) - \Phi(X_n; \theta) \right)^2 \right]. \quad (13)$$

Algorithm 3 Hybrid-Now

-
- 1: Poser $\hat{V}_N = g$
 - 2: **for** $n = N - 1, \dots, 0$ **do**
 - 3: Apprendre $\hat{a}_n$ avec la perte (12).
 - 4: Construire des cibles de Bellman one-step.
 - 5: Apprendre $\hat{V}_n = \Phi(\cdot; \hat{\theta}_n)$ par la régression (13).
 - 6: **end for**
-

Le gain de temps est important, car on ne déroule qu'un pas au lieu de simuler jusqu'à N . En contrepartie, une erreur sur $\hat{V}_{n+1}$ influence directement $\hat{a}_n$ et $\hat{V}_n$. Le warm-start temporel recommandé dans l'article - initialiser les poids du temps n avec ceux du temps $n + 1$ - est donc crucial pour stabiliser l'apprentissage.

3.4 Hybrid-LaterQ : regress later et quantification

Hybrid-LaterQ conserve l'étape politique de Hybrid-Now, mais remplace l'espérance par une somme sur un quantifieur du bruit. Soit

$$\hat{\varepsilon} \in \{e_1, \dots, e_K\}, \quad \Pr(\hat{\varepsilon} = e_k) = p_k.$$

Après apprentissage de $\hat{a}_n$, on interpole la valeur suivante :

$$\hat{\theta}_{n+1} \in \arg \min_{\theta} \mathbb{E} \left[\left(\hat{V}_{n+1}(X_{n+1}) - \Phi(X_{n+1}; \theta) \right)^2 \right], \quad \tilde{V}_{n+1} = \Phi(\cdot; \hat{\theta}_{n+1}). \quad (14)$$

Puis on calcule

$$\hat{V}_n(x) = f(x, \hat{a}_n(x)) + \sum_{k=1}^K p_k \tilde{V}_{n+1}(F(x, \hat{a}_n(x), e_k)). \quad (15)$$

Le bénéfice est une variance Monte Carlo plus faible ; la limite est l'erreur de quantification, surtout si K est faible ou si le bruit est multidimensionnel.

3.5 Qknn : quantification et plus proches voisins

Qknn est une méthode sans réseau. On construit des grilles d'état Γ_n , une quantification du bruit, puis

$$\hat{Q}_n(z, a) = f(z, a) + \sum_{k=1}^K p_k \hat{V}_{n+1} \left(\text{Proj}_{\Gamma_{n+1}}(F(z, a, e_k)) \right), \quad z \in \Gamma_n. \quad (16)$$

On pose ensuite

$$\hat{a}_n(z) \in \arg \min_{a \in A} \hat{Q}_n(z, a), \quad \hat{V}_n(z) = \hat{Q}_n(z, \hat{a}_n(z)).$$

En dehors de la grille, le code utilise une interpolation par voisins. Cette méthode fonctionne très bien en faible dimension, comme le stockage de gaz, mais la taille des grilles croît trop vite avec d .

3.6 ClassifHybrid : contrôle mixte du micro-réseau

Le micro-réseau a un contrôle mixte : soit le générateur est éteint, soit il est allumé avec une production continue dans $[A_{\min}, A_{\max}]$. L'algorithme hybride apprend donc une probabilité d'extinction $p_0(x; \beta^0)$ et une production $\pi(x; \beta^1)$. La perte compare implicitement deux branches :

$$a = 0 \quad \text{ou} \quad a = \pi(x; \beta^1).$$

Cette architecture est plus proche de la structure économique du problème : décision on/off d'abord, niveau de production ensuite.

3.7 Comparaison critique des méthodes

Table 1 – Algorithmes étudiés : type de contrôle traité, principaux avantages et principales limites.

Méthode	Contrôle	Avantages	Limites
NNContPI	Continu	Apprend directement la politique ; pas besoin d’approximer explicitement V_n .	Coût élevé : la perte simule de n à N . Sensible aux politiques futures déjà apprises.
ClassifPI	Discret	Très naturel pour actions finies ; sortie probabiliste interprétable.	Peut lisser les frontières ; nécessite assez d’échantillons près des zones de changement d’action.
Hybrid-Now	Continu ou discret	Rapide ; n’utilise qu’un pas de dynamique.	Propagation des erreurs de valeur ; sensible au warm-start et au choix de réseau.
Hybrid-LaterQ	Continu	Réduit la variance de l’espérance par quantification.	Erreur de quantification et interpolation ; moins adapté si le bruit est de grande dimension.
Qknn	Faible dimension	Benchmark très solide ; pas d’optimisation non convexe de réseau.	Malédiction de la dimension ; qualité dépend fortement des grilles.
ClassifHybrid	Mixte	Respecte la structure on/off + continu.	Plus complexe à entraîner ; deux erreurs couplées, classification et contrôle continu.

4 Méthodologie expérimentale

4.1 Modes d’exécution et conséquences

Le code peut être lancé avec plusieurs niveaux de budget numérique. Le mode **fast**, activé par défaut dans les scripts principaux et dans `run_all.py`, correspond à une version allégée des expériences : il permet d’obtenir rapidement des figures et de vérifier qualitativement les comportements. Le mode complet s’obtient avec l’option `-no-fast` et correspond au réglage le plus proche des expériences complètes. Nous ne l’avons toutefois pas utilisé pour produire les résultats de ce rapport, faute de ressources de calcul suffisantes.

Un mode **smoke** existe seulement pour certaines expériences, notamment `exp4_gas_storage.py` et `exp5_microgrid.py`. Il sert à vérifier rapidement que ces scripts s’exécutent correctement, avec très peu de trajectoires et peu d’itérations. Le script `scripts/smoke_test.py` fournit en plus un test global minimal des imports et de la construction des problèmes.

Les résultats présentés dans ce rapport proviennent du mode **fast**. Ce choix a plusieurs conséquences :

- nombre de trajectoires Monte Carlo inférieur au papier ;
- nombre d’époques et de mini-batches réduit ;
- dimension parfois réduite, notamment $d = 10$ pour la PDE au lieu de $d = 100$;
- quantification plus grossière ;
- grilles Qknn plus petites que les grilles soigneusement construites dans le papier.

Ainsi, plusieurs écarts de résultats avec le papier sont attendus.

4.2 Intervalles de confiance

Par la suite, quand on fournit une moyenne $\bar{X}$, un écart-type empirique s et un nombre de trajectoires M , on utilise l’approximation

$$IC_{95\%} \simeq \bar{X} \pm 1.96 \frac{s}{\sqrt{M}}.$$

Cette approximation est pertinente pour une moyenne Monte Carlo.

5 Expérience 1 - PDE semi-linéaire

5.1 Modèle continu et discret

La première expérience porte sur une équation de Hamilton–Jacobi–Bellman semi-linéaire avec croissance quadratique dans le gradient :

$$\partial_t v + \Delta_x v - |D_x v|^2 = 0, \quad v(T, x) = g(x). \quad (17)$$

Comme

$$-|p|^2 = \inf_{a \in \mathbb{R}^d} (|a|^2 + 2a \cdot p),$$

elle correspond au problème de contrôle

$$v(t, x) = \inf_{\alpha} \mathbb{E} \left[\int_t^T |\alpha_s|^2 ds + g(X_T^{t,x,\alpha}) \right], \quad dX_s = 2\alpha_s ds + \sqrt{2} dW_s. \quad (18)$$

Après discrétisation avec pas $h = T/N$,

$$X_{n+1} = X_n + 2h\alpha_n + \sqrt{2h}\varepsilon_{n+1}, \quad J(\alpha) = \mathbb{E} \left[\sum_{n=0}^{N-1} h|\alpha_n|^2 + g(X_N) \right].$$

La transformation de Hopf–Cole donne une référence :

$$v(t, x) = -\log \mathbb{E} \left[\exp\{-g(x + \sqrt{2}W_{T-t})\} \right]. \quad (19)$$

Dans cette expérience, la fonction g représente le coût terminal. Elle détermine la zone de l’espace d’état vers laquelle le contrôle a intérêt à ramener le processus à l’approche de l’échéance. Deux choix de coûts terminaux sont utilisés dans nos reproductions. Pour le test en grande dimension, on considère le coût régulier

$$g(x) = \log \left(\frac{1 + |x|^2}{2} \right),$$

comme dans le test principal du papier. Ce coût favorise des états proches de l’origine, car il augmente avec la norme de x . Pour le test en dimension un, utilisé dans le balayage en γ , on considère le coût terminal

$$g_\gamma(x) = -x^\gamma \mathbf{1}_{0 \leq x \leq 1} - \mathbf{1}_{x > 1}.$$

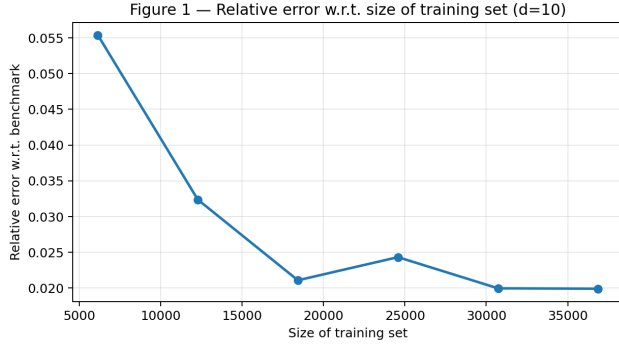
5.2 Notre modèle de reproduction

Dans notre implémentation, cette expérience sert surtout de test de cohérence pour les réseaux de neurones en dimension élevée. On simule le schéma d’Euler de la dynamique contrôlée : à chaque date, le réseau prend l’état courant en entrée et renvoie un contrôle continu. Le coût courant pénalise les contrôles trop forts, tandis que le coût terminal incite le processus à revenir vers une zone favorable. On s’attend donc à une politique assez naturelle : agir peu au début, puis corriger davantage la trajectoire lorsque l’échéance approche.

Les écarts avec le papier s’expliquent principalement par le budget numérique. Nos résultats proviennent d’un mode rapide, avec moins de trajectoires Monte Carlo, moins d’époques d’entraînement et parfois une dimension plus faible que dans l’article. Les mesures d’entraînement sont aussi plus simples : elles couvrent une région raisonnable de l’espace d’état, mais ne reproduisent pas toute la finesse de l’exploration du papier. Pour Hybrid-Now, les réseaux utilisés sont des perceptrons multicouches avec activation ELU. Pour la Figure 1, les réseaux de politique et de valeur ont trois couches cachées de 64 neurones. Pour les trajectoires en dimension 2, l’architecture est plus légère, avec deux couches de 32 neurones. Pour le balayage en γ , le code utilise trois couches cachées de tailles (10, 5, 5). La sortie de la politique est linéaire, car le contrôle est continu et non borné dans ce test.

5.3 Test grande dimension : erreur relative

Le papier considère $d = 100$ et rapporte une erreur relative finale proche de 0.11% pour Hybrid-Now après un budget d’entraînement important. Notre code a été exécuté en mode rapide avec une dimension et un budget plus modestes. La figure suivante met directement côte à côte la figure originale et notre reproduction.



(a) **Reproduction — erreur relative.** Erreur relative de la valeur en fonction du budget d'apprentissage.

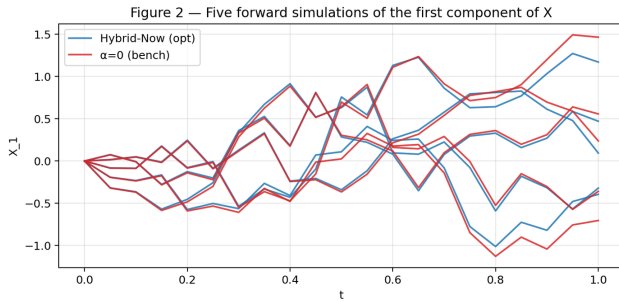


(b) **Papier — erreur relative.** Figure de référence de l'article pour l'erreur relative.

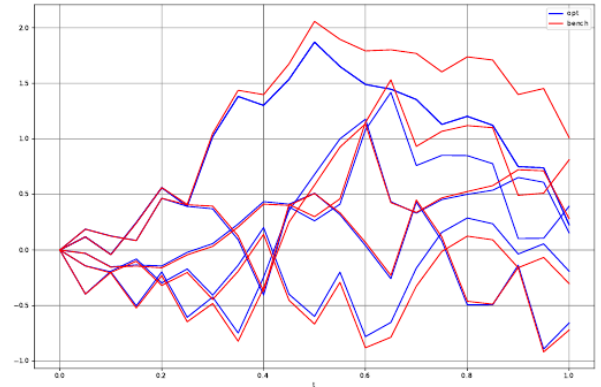
Figure 1 – Erreur relative de la valeur — PDE semi-linéaire. La tendance décroissante de l'erreur est bien retrouvée : l'apprentissage améliore la valeur lorsque le budget de simulation augmente. En revanche, notre courbe se stabilise plus haut que celle du papier. La différence est cohérente avec le mode rapide : moins d'époques, moins de trajectoires, moins de pré-entraînement temporel et une dimension qui n'est pas toujours celle du test publié.

La lecture de cette figure est donc double. La méthode apprend dans la bonne direction, mais la précision publiée n'est pas reproduite avec le budget local. Ce test doit donc être interprété comme une validation de tendance et d'ordre de grandeur, non comme une reproduction exacte de l'article.

5.4 Trajectoires en dimension 2



(a) **Reproduction — trajectoires contrôlées.** Évolution du premier état sous politique apprise et sous stratégie nulle.



(b) **Papier — trajectoires contrôlées.** Figure de référence de l'article pour les trajectoires.

Figure 2 – Trajectoires contrôlées vs stratégie nulle — PDE semi-linéaire. Dans les deux graphes, la politique apprise agit surtout près de l'échéance pour ramener l'état vers une région de coût terminal plus faible, alors que la politique nulle laisse davantage diffuser le processus. Les trajectoires exactes ne coïncident pas (ce qui est normal car elles dépendent des tirages Monte Carlo) mais la tendance dynamique est la même.

Le coût moyen est de 0.358 pour la politique apprise, contre 0.628 pour $\alpha = 0$. L'écart-type trajectoriel est élevé, mais la différence de moyenne est suffisamment grande pour conclure que la politique apprise est meilleure que la stratégie naïve dans cette exécution.

5.5 Balayage en γ

Le second test utilise le coût terminal non lisse g_γ défini plus haut, en dimension 1. Le Tableau 2 compare nos sorties à la Table 1 du papier.

Table 2 – PDE semi-linéaire en dimension 1 : estimation de la valeur initiale $V(0,0)$ selon le paramètre γ , *comparaison entre nos résultats, ceux du papier et la référence Hopf – Cole*.

γ	Valeur du code			Valeur du papier			Référence code Hopf–Cole
	Hybrid-Now	Hybrid-LaterQ	Qknn	Hybrid-Now	Hybrid-LaterQ	Qknn	
1.0	-0.442	-0.493	-0.483	-0.460	-0.456	-0.461	-0.464
0.5	-0.483	-0.525	-0.532	-0.507	-0.495	-0.508	-0.509
0.1	-0.559	-0.576	-0.609	-0.579	-0.572	-0.581	-0.585
0.0	-0.579	-0.592	-0.640	-1.000	-1.000	-1.000	-1.

Pour $\gamma = 1$, $\gamma = 0.5$ et 0.1 , les ordres de grandeur sont corrects.

5.6 Comparaison critique des approches utilisées

Les résultats montrent d’abord que la politique apprise domine bien la stratégie naïve $\alpha = 0$ sur le test en grande dimension : le coût moyen baisse de 0.628 à 0.358. Le gain est donc net au niveau de la moyenne, même si la variance des trajectoires reste élevée.

Sur le balayage en γ , l’analyse est plus contrastée. Pour $\gamma = 1$, les trois méthodes retrouvent le bon ordre de grandeur, avec un léger avantage pour Qknn. Pour $\gamma = 0.5$ et $\gamma = 0.1$, c’est plutôt Hybrid-LaterQ qui est la plus proche de la référence. En revanche, Hybrid-Now est globalement moins précise sur ces tests 1D.

Le point vraiment important est le cas $\gamma = 0$. Aucune des trois méthodes ne retrouve la valeur de référence -1 , avec un écart encore important. Cela montre que notre implémentation reste fragile dès que le coût terminal devient très peu régulier.

En résumé, nos expériences valident bien le comportement qualitatif attendu : les méthodes apprennent une politique utile et retrouvent des valeurs plausibles pour $\gamma > 0$. En revanche, la précision quantitative reste inégale selon les cas, et le test $\gamma = 0$ montre clairement que nous ne reproduisons pas encore complètement le niveau de performance du papier.

6 Expérience 2 - Contrôle linéaire-quadratique

6.1 Modèle et solution de référence

Le modèle continu est

$$dX_t = (BX_t + C\alpha_t) dt + \sum_{j=1}^p D_j \alpha_t dW_t^j. \quad (20)$$

Le coût est quadratique :

$$\mathbb{E} \left[\int_t^T (X_s^\top Q X_s + \lambda |\alpha_s|^2) ds + X_T^\top P X_T \right].$$

La valeur exacte est $v(t, x) = x^\top K(t)x$, où $K(t)$ résout une équation de Riccati. Le contrôle optimal est linéaire en x :

$$a^*(t, x) = - \left(\lambda I_m + \sum_{j=1}^p D_j^\top K(t) D_j \right)^{-1} C^\top K(t)x.$$

Cette expérience est utile car elle permet de vérifier à la fois la forme de la politique et celle de la valeur.

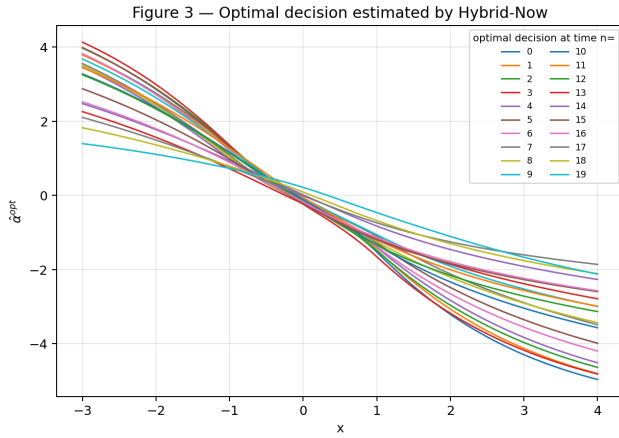
6.2 Notre modèle de reproduction

Notre code simule la version discrète du problème LQ, c’est-à-dire un problème de contrôle linéaire-quadratique. Le terme « linéaire » vient de la dynamique de l’état, qui dépend linéairement de l’état X_n et du contrôle α_n . Le terme « quadratique » vient de la fonction de coût, qui pénalise quadratiquement la taille de l’état et l’intensité du contrôle. On compare ensuite les résultats obtenus aux valeurs de Riccati, qui fournissent la solution de référence.

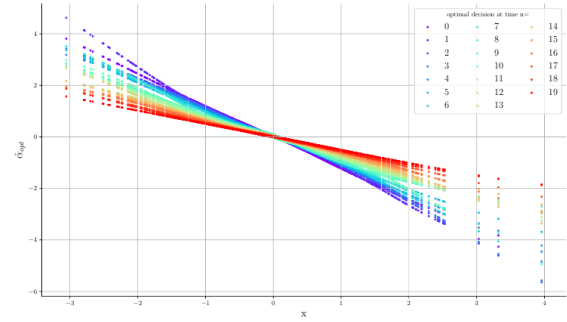
Dans cette expérience, l’état est vectoriel et le contrôle est continu. L’intérêt du test est que la théorie impose une forme très précise : la politique optimale doit être affine en l’état, et la fonction valeur doit être quadratique. Ainsi, même avant de comparer les valeurs numériques, les graphes permettent de détecter rapidement une implémentation incorrecte.

Les réseaux LQ sont volontairement simples, car la solution exacte est régulière. Pour $d \leq 20$, le code utilise deux couches cachées de tailles $(d + 20, d + 10)$, donc $(21, 11)$ en dimension 1 et $(30, 20)$ en dimension 10. Pour des dimensions plus grandes, l’architecture est plafonnée à $(64, 64)$. L’activation est ELU, la sortie de la politique est linéaire, et la valeur est une sortie scalaire. En mode fast, Adam utilise 15 époques, 8 mini-batches par époque, batch 256, learning-rate $5 \cdot 10^{-3}$ et régularisation $L^2 = 10^{-4}$. Cette architecture explique pourquoi on retrouve bien les formes linéaire et quadratique, même si les valeurs finales restent sensibles à la discrétisation.

6.3 Résultats graphiques et numériques

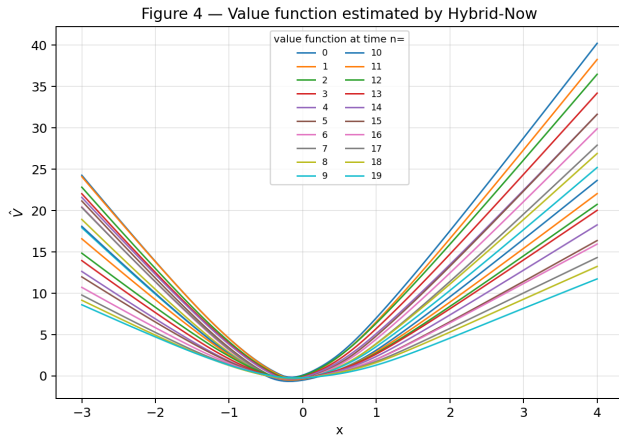


(a) **Reproduction** — **politique LQ**. Contrôle optimal estimé en fonction de l'état.

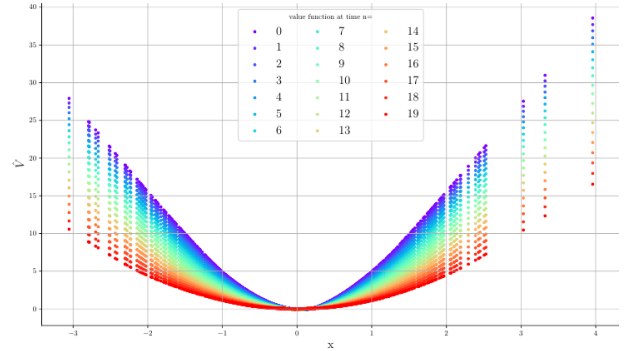


(b) **Papier** — **politique LQ**. Figure de référence de l'article pour le contrôle optimal.

Figure 3 – Politique optimale en fonction de l'état — contrôle LQ. La politique apprise dans notre code a bien une forme presque affine en l'état, ce qui est conforme à la solution de Riccati. Les pentes et les niveaux ne sont pas exactement identiques, mais la tendance principale est correctement reproduite.



(a) **Reproduction** — **valeur LQ**. Fonction de valeur estimée en fonction de l'état.



(b) **Papier** — **valeur LQ**. Figure de référence de l'article pour la fonction de valeur.

Figure 4 – Fonction de valeur en fonction de l'état — contrôle LQ. La fonction valeur reproduite conserve la convexité et la forme quadratique attendues. C'est un test qualitatif important : si la valeur apprise n'était pas parabolique, cela indiquerait que le réseau ou la perte ne respecte pas la structure du problème.

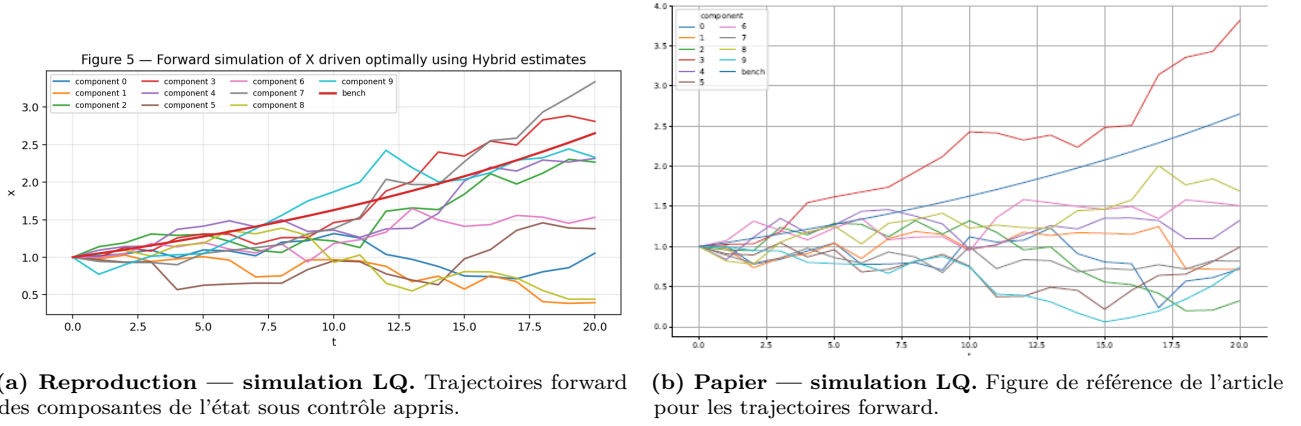


Figure 5 — Trajectoires forward de l'état — contrôle LQ. Les trajectoires contrôlées ne peuvent pas coïncider trajectoire par trajectoire, mais le comportement global est cohérent : le contrôle réduit progressivement la norme de l'état. Cette tendance est la même que dans le papier.

Table 3 — Contrôle linéaire-quadratique en dimension $d = 10$: estimation de la valeur initiale $V(0, X_0)$ pour $X_0 = \mathbf{1}_{10}$. La référence Riccati du code est discrète ; celle du papier est continue et calculée avec Matlab.

Méthode	Valeur du code	Valeur du papier	Écart code - papier
Hybrid-Now	57.37	56.0	+1.37
NNContPI	56.50	54.3	+2.20
Riccati	54.28	57.1	-2.82

Les valeurs de Hybrid-Now et NNContPI sont proches de l'échelle du papier. Le fait que NNContPI ne soit pas strictement meilleur que Hybrid-Now dans notre table, alors que le papier le présente comme légèrement plus précis sur certains cas, peut venir du budget rapide et du bruit d'optimisation. Le point important est que la structure linéaire/quadratique est retrouvée.

6.4 Comparaison critique des approches utilisées

La référence Riccati est ici la méthode de contrôle : elle exploite la structure quadratique et donne la meilleure information disponible sur la forme attendue. NNContPI est conceptuellement adapté, car il optimise directement une politique continue et peut capturer une politique linéaire sans imposer explicitement la forme. En revanche, il est coûteux : à un temps donné, il doit propager les trajectoires sous les politiques futures déjà apprises. Hybrid-Now est plus rapide, car il remplace une partie de cette propagation par une approximation de la fonction valeur, mais il dépend alors de la qualité de cette régression.

Sur nos résultats, les deux méthodes neurales retrouvent l'allure qualitative imposée par la théorie. La comparaison numérique est plus délicate : NNContPI n'est pas systématiquement meilleur dans cette exécution, alors que le papier observe parfois une meilleure précision. Cet écart est compatible avec le mode rapide et le bruit d'entraînement. On ne peut donc pas classer définitivement Hybrid-Now et NNContPI à partir de cette seule exécution ; on retient plutôt que les deux méthodes respectent la structure LQ, avec un compromis temps de calcul/précision différent.

7 Expérience 3 - Couverture quadratique d'option

7.1 Modèle

On considère un investisseur qui détient une quantité α_n d'un actif risqué de prix P_n . La richesse vérifie

$$W_{n+1} = W_n + \alpha_n R_{n+1},$$

où R_{n+1} est le rendement. Pour un payoff $h(P_N)$, le problème est

$$\inf_{\alpha} \mathbb{E} \left[(h(P_N) - W_N^{\alpha})^2 \right].$$

Dans le test, $N = 6$, $P_0 = 100$, $h(p) = (p - 100)^+$, et les rendements suivent un modèle trinomial. La solution exacte de Bertsimas-Kogan-Lo donne un prix de couverture quadratique d'environ 4.62.

7.2 Notre modèle de reproduction

Le modèle reproduit est un problème de couverture quadratique en marché incomplet. L'état contient la richesse courante et le prix de l'actif risqué. Le contrôle est la quantité d'actif détenue pendant la période suivante. Le critère n'est pas de maximiser une richesse moyenne, mais de minimiser l'erreur quadratique entre la richesse terminale et le payoff du call. Cette formulation est importante : une politique peut donner un prix initial plausible tout en produisant une mauvaise couverture trajectorielle, car la perte dépend de la dispersion de l'erreur terminale.

Notre reproduction conserve le modèle trinomial du papier et compare les réseaux au benchmark de Bertsimas–Kogan–Lo. C'est aussi l'expérience où le choix d'architecture est le plus sensible. Le papier recommande d'exploiter la structure linéaire-quadratique : la politique est affine en la richesse et la fonction valeur est quadratique en la richesse. Si l'implémentation utilise un réseau trop générique ou un entraînement trop court, le minimum de la courbe de valeur peut être mal localisé, ce qui explique les prix neuraux trop éloignés du benchmark dans l'expérience présentée.

Le code tient compte de cette structure au moyen de deux classes spécialisées. `HedgingPolicyNet` ne traite pas toute la paire (W, P) comme une entrée indifférenciée : il fait passer le prix P dans un MLP, puis produit deux coefficients qui définissent une politique affine en richesse, $a(W, P) = b_0(P) + b_1(P)W$. De même, `HedgingValueNet` produit trois coefficients dépendant du prix et impose une valeur quadratique en richesse, $V(W, P) = c_0(P) + c_1(P)W + c_2(P)W^2$, avec $c_2 > 0$ via une `softplus`. En mode rapide, le MLP a deux couches cachées de 32 neurones, activation ELU, 80 époques, 20 mini-batches par époque, batch 1024, learning-rate 10^{-3} , régularisation 10^{-6} et clipping du gradient à 1. Malgré cet ansatz, l'expérience reste très sensible à la qualité de l'entraînement.

7.3 Résultats graphiques et numériques

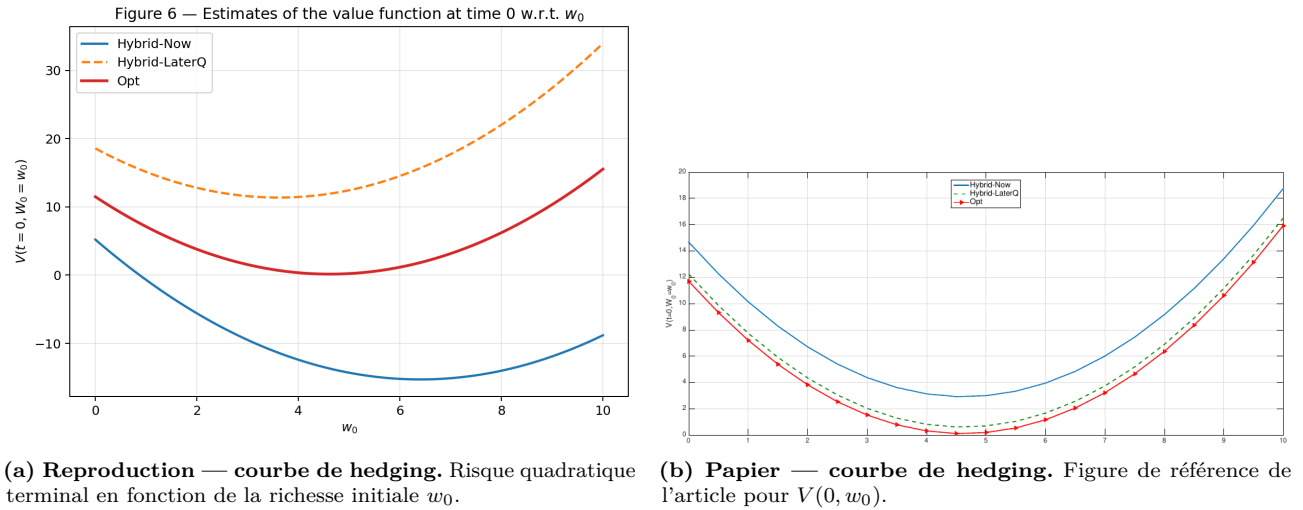
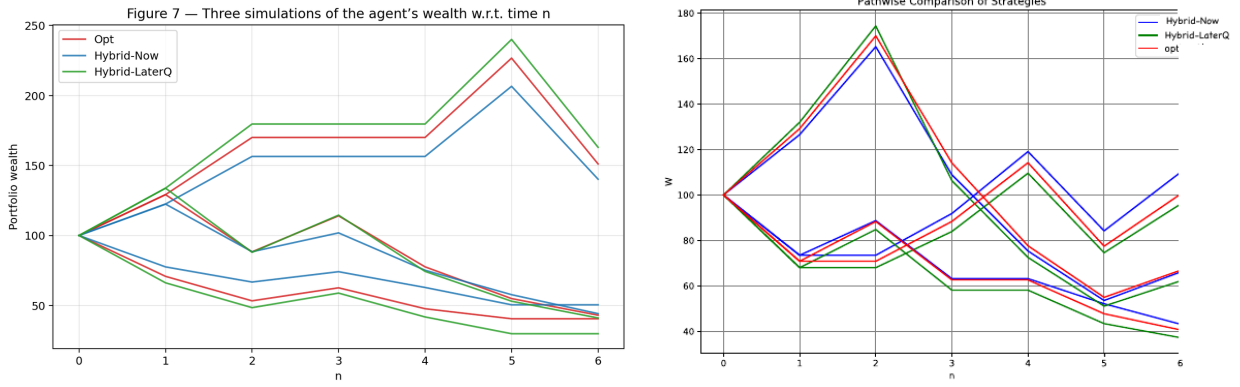


Figure 6 — Couverture quadratique d'option. Le papier obtient des courbes plus proches du benchmark exact. Dans notre reproduction, les courbes neurales sont plus éloignées, ce qui confirme que cette expérience doit être présentée comme une reproduction qualitative et non comme une reproduction quantitative complète.

Table 4 — Couverture quadratique d'option : prix initial estimé w_0^* et perte quadratique terminale moyenne $E[(h(P_N) - W_N)^2]$. $L'IC95$ est calculé avec $M = 5000$ trajectoires à partir de l'écart-type empirique.

Méthode	Prix initial estimé	Perte quadratique moyenne	IC95 sur la perte
Exact BKL	4.62	0.104	± 0.014
Hybrid-Now	6.40	27.02	± 1.50
Hybrid-LaterQ	3.61	28.34	± 1.99
Papier	≈ 4.5	non tabulé	-



(a) **Reproduction — trajectoires de richesse.** Évolution de la richesse sous différentes stratégies de couverture.

(b) **Papier — trajectoires de richesse.** Figure de référence de l'article pour les trajectoires de richesse.

Figure 7 — Couverture quadratique d'option. La structure de la figure est retrouvée, mais les écarts visibles entre stratégies justifient les réserves formulées dans la comparaison numérique. La tendance du papier, où les trajectoires neurales suivent mieux la stratégie optimale, est moins nette dans cette exécution.

Cette expérience demande le plus de recul critique. Le benchmark exact est cohérent avec le papier, mais les réseaux entraînés produisent des pertes très supérieures. La reproduction est donc partielle : elle est surtout graphique, et elle ne doit pas être présentée comme quantitative, même si certaines tendances globales sont visibles. Les causes probables sont : entraînement trop court, forte sensibilité du minimum de la courbe de valeur, variance Monte Carlo importante dans l'évaluation de la perte, et optimisation non convexe des réseaux. Une amélioration naturelle consisterait à relancer l'entraînement avec plusieurs graines, augmenter le budget numérique et comparer des courbes de valeur moyennées autour du minimum.

7.4 Comparaison critique des approches utilisées

Le benchmark BKL est indispensable dans cette expérience : il donne à la fois un prix de référence et une structure attendue pour la stratégie de couverture. Hybrid-Now et Hybrid-LaterQ ont des objectifs proches, mais pas la même source d'erreur. Hybrid-Now dépend fortement de la qualité de la régression de la valeur au pas suivant ; il peut donc être rapide mais instable si la courbe de valeur est mal apprise près de son minimum. Hybrid-LaterQ remplace une partie de l'espérance conditionnelle par une étape de régression-later et de quantification ; cela peut réduire la variance, mais introduit une erreur de quantification et une dépendance forte au choix de la grille.

Dans cette exécution, aucune des deux approches neurales ne reproduit quantitativement la perte du benchmark. Hybrid-Now surestime le prix de couverture, tandis que Hybrid-LaterQ le sous-estime. Le fait que les deux pertes terminales soient beaucoup plus élevées que la perte exacte indique que le problème n'est pas seulement une mauvaise lecture du prix, mais bien une politique de couverture insuffisamment entraînée. Cette expérience doit donc être présentée comme un cas d'échec partiel utile : même avec l'ansatz affine/quadratique, le résultat reste sensible au budget d'entraînement et à la localisation du minimum de la courbe de valeur.

8 Expérience 4 - Stockage de gaz

8.1 Modèle

L'état est (P_n, C_n) , où P_n est le prix du gaz et C_n le niveau de stock. Le prix suit un processus de retour à la moyenne :

$$P_{n+1} = \bar{p}(1 - \beta) + \beta P_n + \xi_{n+1}.$$

Le contrôle appartient à $\{-1, 0, 1\}$: retirer, attendre ou injecter selon la convention. Le niveau de stock reste dans $[C_{\min}, C_{\max}]$. La récompense courante dépend du prix, du volume injecté ou retiré, et de pertes physiques. Le papier compare Hybrid-Now, ClassifPI et Qknn pour plusieurs valeurs de a_{in} .

8.2 Notre modèle de reproduction

La reproduction utilise le même type d'état bidimensionnel que le papier : prix du gaz et niveau de stockage. Le contrôle est discret, mais l'admissibilité dépend du niveau de stock : près de la capacité maximale, injecter n'est plus possible ; près de la borne basse, retirer n'est plus possible. Le modèle est donc un bon test pour comparer une méthode de classification, une méthode hybride plus générale et une méthode de grille.

Dans le code, Qknn sert de référence basse dimension. Il utilise une grille en (P, C) , une quantification du bruit et une recherche locale de l'action optimale. Pour ClassifPI, le réseau est un MLP à deux couches cachées

de 20 neurones avec activation ELU. La couche de sortie contient trois logits, correspondant aux trois actions, puis une softmax donne les probabilités. Le code masque les actions non admissibles près des bornes de stock, ce qui évite de sélectionner une injection ou un retrait impossible. Pour Hybrid-Now, l'exécution rapide utilise deux couches de 24 neurones ; le mode long utilise (32, 32, 16). Les deux configurations utilisent Adam, warm-start temporel et régularisation L^2 . En mode rapide, ClassifPI utilise 6 époques, 50 mini-batches par époque, batch 768 et learning-rate $8 \cdot 10^{-4}$; Hybrid-Now utilise 5 époques, 52 mini-batches, batch 768 et learning-rate $3 \cdot 10^{-4}$. Ces budgets restent modestes par rapport aux frontières très nettes du problème, ce qui explique les écarts de ClassifPI et Hybrid-Now face à Qknn.

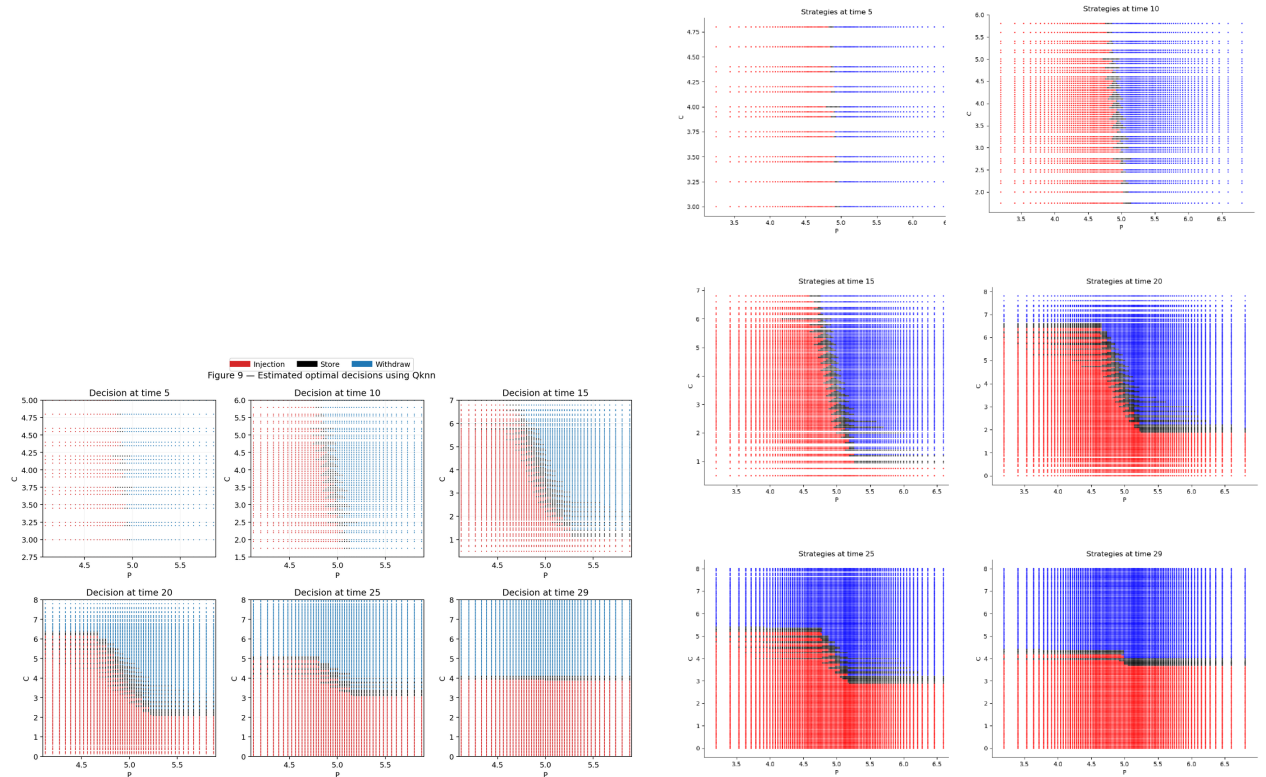
8.3 Comparaison table à table

Table 5 — Stockage de gaz : valeur initiale estimée du problème selon le taux d'injection a_{in} , comparaison entre notre code et la Table 4 du papier.

a_{in}	Valeur du code				Valeur du papier			
	Hybrid-Now	ClassifPI	Qknn	$\alpha = 0$	Hybrid-Now	ClassifPI	Qknn	$\alpha = 0$
0.06	-0.80	-1.11	-0.66	-1.20	-0.99	-0.71	-0.66	-1.20
0.10	-0.90	-1.30	-0.34	-1.20	-0.70	-0.38	-0.34	-1.20
0.20	-0.74	-0.84	0.12	-1.20	-0.21	0.01	0.12	-1.20
0.30	-0.36	-0.75	0.37	-1.20	-0.10	0.37	0.37	-1.20
0.40	-0.12	-1.02	0.69	-1.20	0.10	0.51	0.69	-1.20

Cette expérience illustre très bien la différence entre méthodes. Qknn reproduit la table du papier à l'arrondi près. Cela indique que la dynamique, la récompense et l'évaluation de Qknn sont bien alignées avec l'article. À l'inverse, ClassifPI est systématiquement trop bas, surtout pour $a_{in} = 0.3$ et 0.4 . Hybrid-Now est instable, ce qui est cohérent avec le papier : les récompenses sont discontinues par rapport au contrôle.

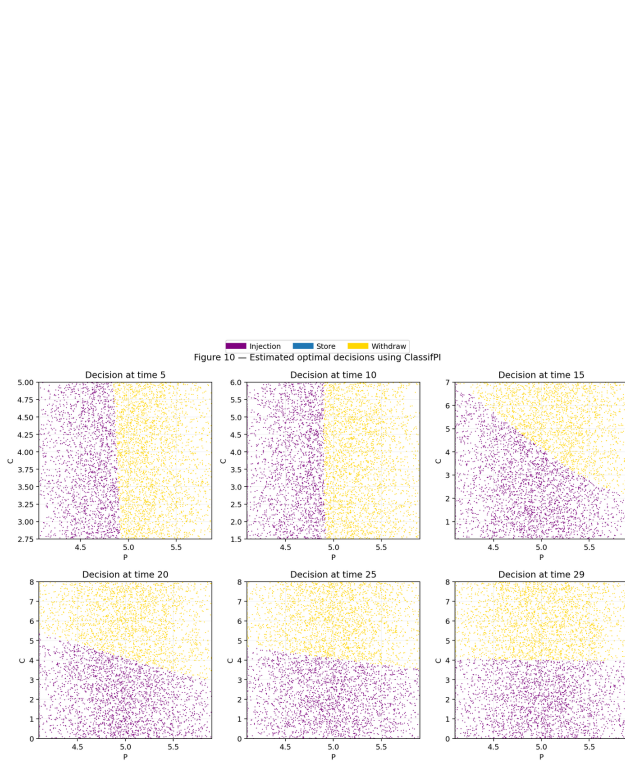
8.4 Résultats graphiques



(a) Reproduction — politique gaz Qknn. Régions de décision injection / attente / retrait selon le prix et le stock.

(b) Papier — politique gaz Qknn. Figure de référence de l'article pour les régions de décision.

Figure 8 — Régions de décision prix-stock — stockage de gaz, méthode Qknn. La frontière de décision et l'organisation des zones d'action sont très proches du papier. Cette comparaison graphique est la plus convaincante de cette expérience, et elle est cohérente avec la reproduction quasi exacte de la table numérique.

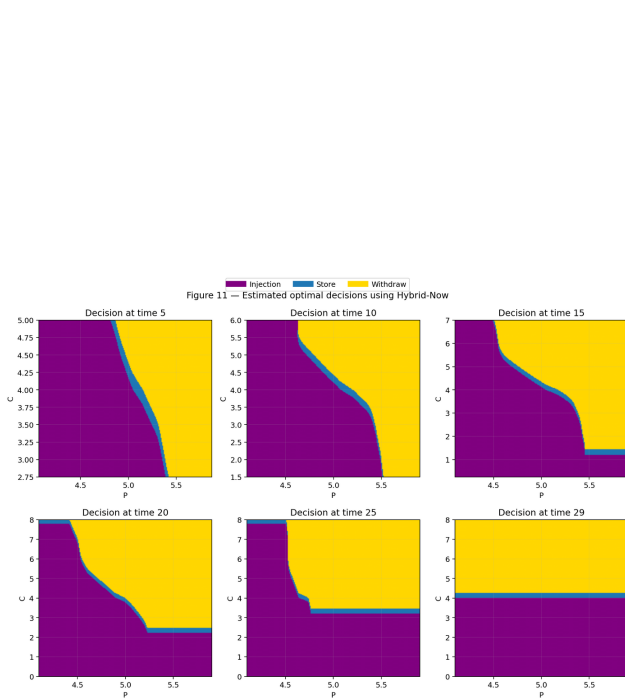


(a) **Reproduction — politique gaz ClassifPI.** Régions de décision injection / attente / retrait selon le prix et le stock.

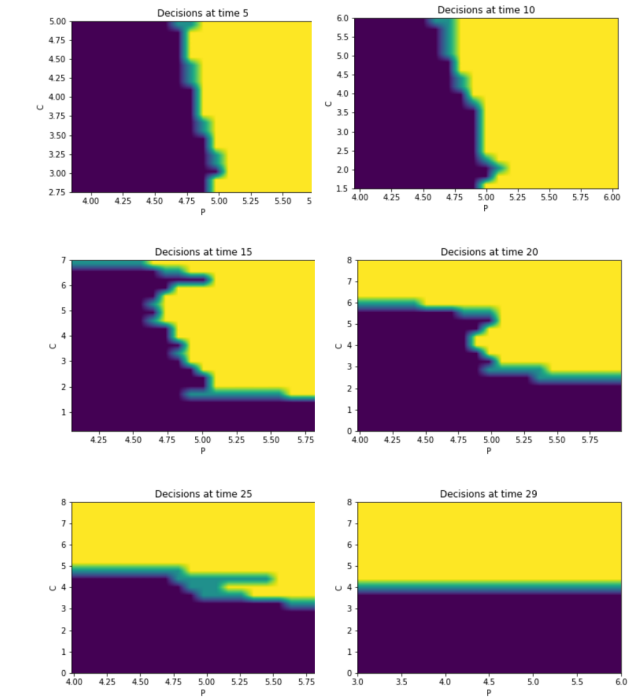


(b) **Papier — politique gaz ClassifPI.** Figure de référence de l'article pour les régions de décision.

Figure 9 — Régions de décision prix-stock — stockage de gaz, méthode ClassifPI. La structure économique générale est visible, mais les frontières sont moins propres que dans le papier. Cela correspond aux écarts numériques : une frontière visuellement plausible ne suffit pas à obtenir une bonne valeur.



(a) **Reproduction — politique gaz Hybrid-Now.** Régions de décision injection / attente / retrait selon le prix et le stock.



(b) **Papier — politique gaz Hybrid-Now.** Figure de référence de l'article pour les régions de décision.

Figure 10 — Régions de décision prix-stock — stockage de gaz, méthode Hybrid-Now. La structure générale est visible, mais les décisions sont plus instables. Cette instabilité est cohérente avec la difficulté du problème pour une méthode continue généraliste : la récompense change brutalement avec l'action discrète et les contraintes de stock.

Graphiquement, on retrouve l'interprétation économique : injecter quand le prix est faible, retirer quand le prix est élevé, et attendre dans une région intermédiaire. Cependant, une frontière visuellement plausible ne garantit pas une bonne valeur. C'est ce que montre ClassifPI dans nos résultats.

8.5 Comparaison critique des approches utilisées

Qknn est clairement l'approche la plus solide sur ce problème. La dimension est faible et l'action est discrète, donc une méthode de grille bien construite peut exploiter la structure locale du problème. ClassifPI est théoriquement bien adapté puisque le contrôle est discret, mais il demande des frontières de décision très nettes. Si l'entraînement est trop court, la politique peut être correcte visuellement sur certaines régions et mauvaise sur les zones qui contribuent le plus à la valeur. Hybrid-Now est plus général mais moins naturel ici, car le problème n'est pas essentiellement continu.

Le papier conclut que ClassifPI est meilleur que Hybrid-Now mais reste inférieur à Qknn pour capturer certaines frontières triangulaires. Nos résultats confirment seulement la supériorité de Qknn ; ils ne reproduisent pas la performance de ClassifPI. La comparaison critique est donc importante : ce n'est pas le modèle de gaz qui est mal implémenté, car Qknn colle au papier, mais plutôt le budget ou la stabilité de l'entraînement de la classification.

9 Expérience 5 - Gestion d'un micro-réseau

9.1 Modèle

L'état est (C_n, M_n, R_n) , où C_n est la charge de batterie, $M_n \in \{0, 1\}$ indique si le générateur était allumé, et R_n est la demande résiduelle. Le contrôle est mixte :

$$a_n \in \{0\} \cup [A_{\min}, A_{\max}].$$

Le coût pénalise notamment l'énergie non satisfaite, la surproduction et les coûts de démarrage ou fonctionnement. La difficulté algorithmique vient du mélange entre décision discrète et quantité continue.

9.2 Notre modèle de reproduction

Dans notre code, le micro-réseau combine une batterie, un générateur thermique et une demande résiduelle aléatoire. La décision est économique : faut-il allumer le générateur, et si oui quelle puissance produire ? Le modèle est donc plus riche que le stockage de gaz, car l'action n'est ni purement discrète ni purement continue.

L'architecture utilisée est ClassifHybrid. Elle est adaptée à la géométrie du contrôle $0 \cup [A_{\min}, A_{\max}]$: une branche apprend une décision discrète, c'est-à-dire la probabilité de laisser le générateur éteint, et une autre branche apprend la puissance continue à produire lorsque le générateur est allumé. En mode rapide, le réseau a deux couches cachées de 32 neurones, activation ELU, 5 époques, 45 mini-batches par époque, batch 768, learning-rate $8 \cdot 10^{-4}$ et régularisation $L^2 = 10^{-4}$. La sortie continue est bornée dans $[A_{\min}, A_{\max}]$, et les contraintes d'état sont traitées par pénalisation. Qknn ne possède pas de réseau : il utilise des grilles en batterie et demande résiduelle, une quantification du bruit et une recherche locale de l'action.

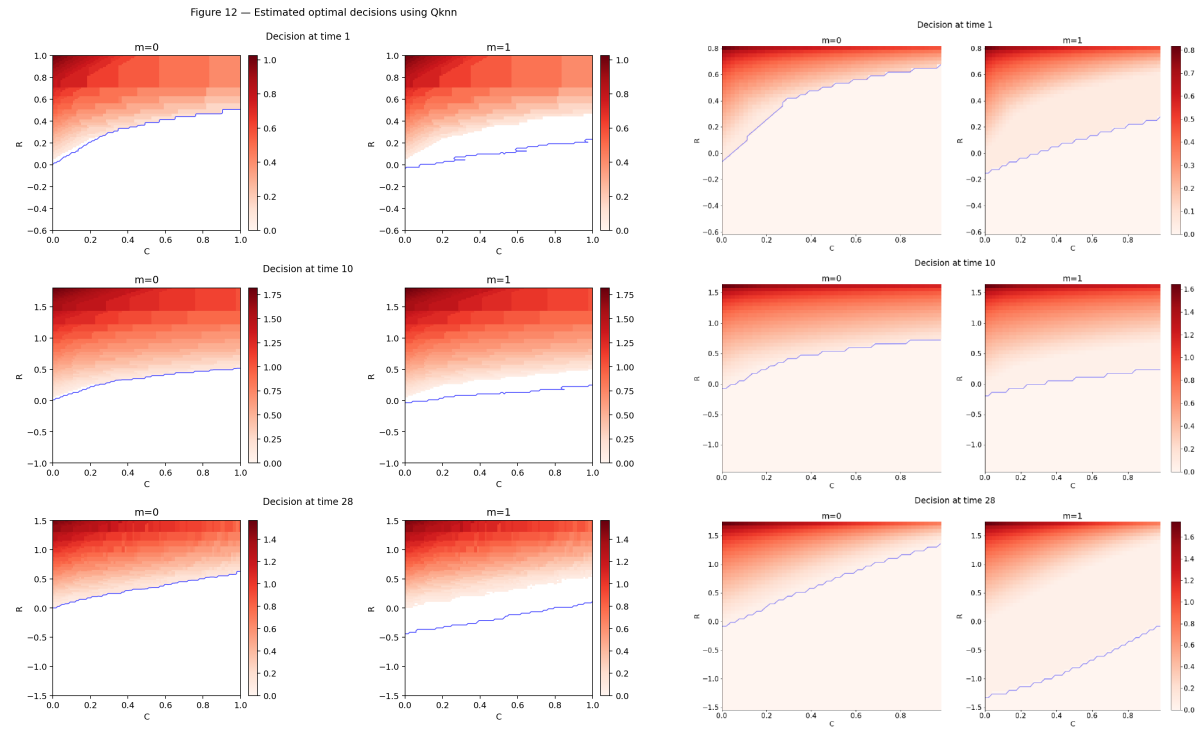
9.3 Comparaison numérique

Table 6 – Micro-réseau : coût total moyen estimé pour $N = 30$, $C_{\max} = 1$ et $(C_0, M_0, R_0) = (0, 0, 0.1)$. L'IC95 du code est calculé en supposant 10 000 trajectoires.

Méthode	Coût moyen du code	IC95 du code	Coût moyen du papier	Écart-type papier
ClassifHybrid	33.07	± 0.64	33.34	0.31
Qknn	32.78	± 0.62	35.37	0.34

La moyenne obtenue avec ClassifHybrid est très proche de celle du papier : 33.07 contre 33.34. Pour Qknn, notre code donne 32.78, soit une valeur plus faible que 35.37. Comme il s'agit d'un problème de minimisation, cela pourrait sembler plus favorable, mais cet écart doit être interprété avec prudence : il peut provenir de différences de grille, de paramétrage ou de protocole d'évaluation par rapport à l'article. Par ailleurs, notre écart-type est calculé sur les trajectoires simulées, tandis que celui du papier semble plutôt correspondre à la variabilité entre répétitions indépendantes de l'expérience ; les deux quantités ne sont donc pas directement comparables. Enfin, nos premiers résultats sur cette expérience étaient nettement moins bons ; nous avons donc augmenté le budget d'entraînement, ce qui a permis d'améliorer sensiblement les performances obtenues.

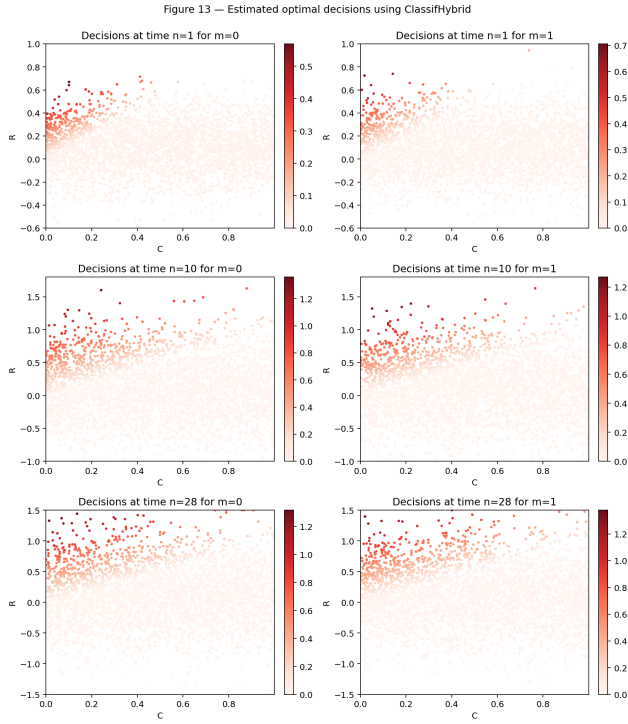
9.4 Résultats graphiques



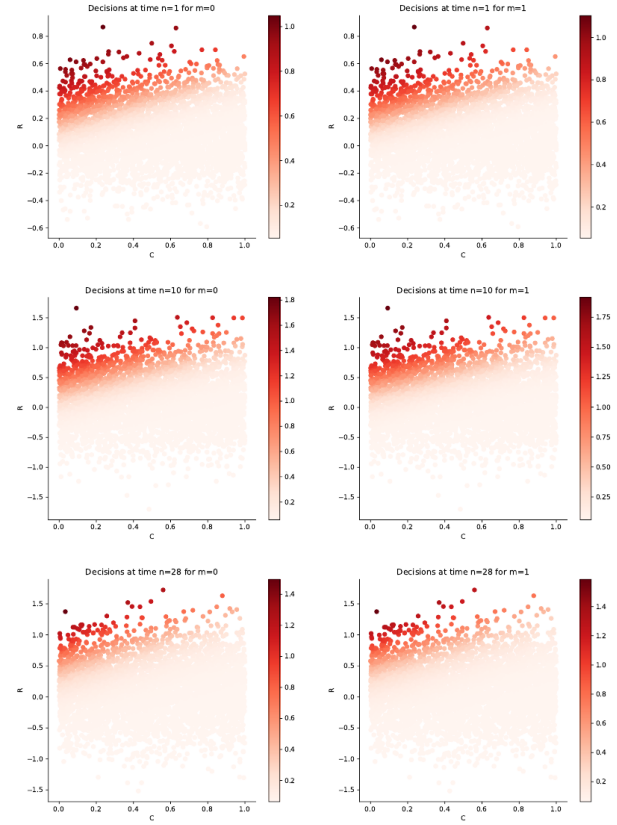
(a) **Reproduction** — politique micro-réseau Qknn. Régions de décision selon la charge de batterie et la demande résiduelle.

(b) **Papier** — politique micro-réseau Qknn. Figure de référence de l'article pour les régions de décision.

Figure 11 — Régions de décision batterie-demande — micro-réseau, méthode Qknn. On retrouve la topologie générale des zones d'action : le générateur est davantage sollicité lorsque la batterie est faible et la demande élevée. Les frontières exactes ne sont pas identiques, ce qui peut venir de la grille et des tirages utilisés.

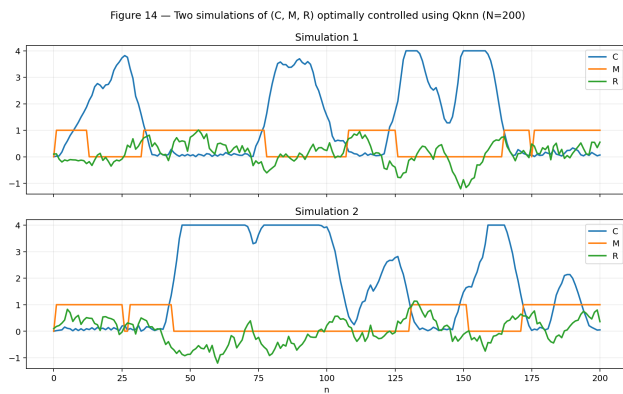


(a) **Reproduction — politique micro-réseau ClassifHybrid.** Régions de décision selon la charge de batterie et la demande résiduelle.

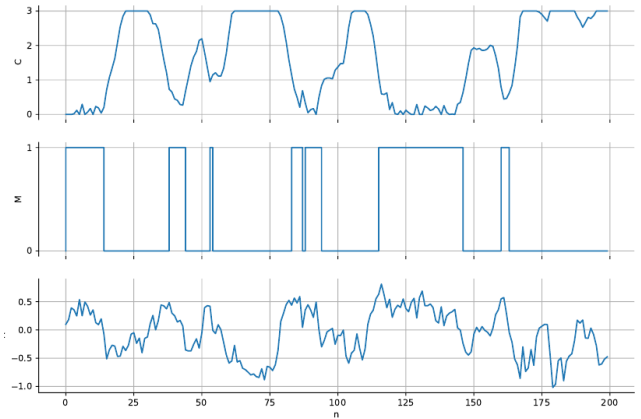


(b) **Papier — politique micro-réseau ClassifHybrid.** Figure de référence de l'article pour les régions de décision.

Figure 12 — Régions de décision batterie-demande — micro-réseau, méthode ClassifHybrid. C'est l'une des comparaisons graphiques les plus convaincantes : la structure des régions de décision est proche du papier, et cela est cohérent avec la proximité des valeurs numériques de la Table 5.



(a) **Reproduction — trajectoires du micro-réseau.** Batterie, générateur et demande résiduelle au cours du temps.



(b) **Papier — trajectoires du micro-réseau.** Figure de référence de l'article pour les trajectoires.

Figure 13 — Trajectoires de batterie, générateur et demande — micro-réseau. Les profils sont cohérents avec ceux de l'article : la batterie se charge et se décharge selon la demande, tandis que le générateur s'active lorsque la batterie ne suffit pas. Les trajectoires exactes diffèrent naturellement d'une réalisation Monte Carlo à l'autre.

Les figures sont qualitativement proches de celles du papier : la frontière de décision dépend de la charge de batterie et de la demande. La bonne performance de ClassifHybrid vient du fait que l'architecture respecte la structure mixte du contrôle.

9.5 Comparaison critique des approches utilisées

ClassifHybrid est l'approche la plus naturelle ici, car elle respecte la forme du contrôle : une décision discrète d'arrêt/allumage et une décision continue de puissance. Une méthode purement classificatoire perdrait l'information sur le niveau de production, tandis qu'une méthode purement continue traiterai mal la masse isolée en zéro. Qknn est utile comme benchmark, mais il dépend fortement de la grille et devient coûteux lorsque la dimension de l'état augmente.

La comparaison numérique favorise ClassifHybrid dans cette exécution : sa moyenne est très proche de la Table 5 du papier. Les figures confirment cette lecture, car les régions de décision sont qualitativement proches. Qknn reste interprétable et robuste, mais l'écart avec le papier est plus important, probablement en raison du choix de grille ou d'une différence d'évaluation. Cette expérience illustre donc l'intérêt d'une architecture adaptée à la structure du contrôle.

10 Bilan critique et robustesse

10.1 Reproduction qualitative vs quantitative

Table 7 – Bilan synthétique par expérience : niveau de reproduction obtenu et principal point évalué.

Expérience	Niveau de reproduction	Point principal évalué
PDE semi-linéaire	qualitative correcte	Erreur décroissante et trajectoires cohérentes ; précision inférieure au papier à cause du mode rapide.
PDE 1D γ	partielle	Bon ordre de grandeur pour $\gamma > 0$, mais cas $\gamma = 0$ non comparable sans vérifier la convention.
LQ	bonne	Structure linéaire/quadratique retrouvée ; attention à la référence Riccati discrète vs continue.
Couverture option	insuffisante quantitativement	Benchmark exact correct, mais pertes neurales trop élevées.
Stockage gaz	excellente pour Qknn, faible pour ClassifPI	Qknn reproduit la Table 4 ; ClassifPI nécessite un réentraînement.
Micro-réseau	bonne	ClassifHybrid proche du papier ; Qknn à interpréter prudemment.

10.2 Sources d'écart

Les écarts viennent de plusieurs facteurs :

1. **Budget Monte Carlo.** Moins de trajectoires augmente la variance des estimations et rend les frontières de décision bruitées.
2. **Optimisation des réseaux.** Les pertes ne sont pas convexes. Deux exécutions avec des graines différentes peuvent donner des politiques différentes.
3. **Dimension et maillage.** Qknn est très sensible à la grille. Une grille trop grossière peut créer des politiques artificiellement bonnes ou mauvaises.
4. **Quantification.** Hybrid-LaterQ et Qknn dépendent du nombre de points K et de la qualité des poids.
5. **Références différentes.** Dans LQ, la Riccati discrète du code n'est pas la Riccati continue du papier.

10.3 Comparaison finale des méthodes

Qknn est la méthode la plus fiable en faible dimension lorsque la grille est bien choisie. C'est visible dans le stockage de gaz. En revanche, elle ne passe pas à la grande dimension. Les réseaux sont alors nécessaires, mais leur réussite dépend beaucoup de l'architecture, de la normalisation, du warm-start et du nombre d'époques. Hybrid-Now est rapide mais peut propager les erreurs ; NNContPI est plus coûteux mais plus direct ; Hybrid-LaterQ réduit la variance au prix d'une erreur de quantification ; ClassifPI est élégant pour les actions discrètes mais sensible aux frontières ; ClassifHybrid est un bon compromis lorsque le contrôle est mixte.

11 Implémentation informatique

11.1 Paramètres neuraux dans les scripts

Les scripts exposent les paramètres neuraux directement dans les fichiers `scripts/exp*.py`. Les configurations importantes sont les suivantes :

- `exp1_semilinear_pde.py` : Hybrid-Now en (64, 64, 64) pour la Figure 1, (32, 32) pour les trajectoires, (10, 5, 5) pour le test en γ ; activation ELU, Adam, $L^2 = 10^{-4}$.
- `exp2_linear_quadratic.py` : deux couches ($d + 20, d + 10$) si $d \leq 20$, sinon (64, 64); activation ELU; politique continue linéaire en sortie.
- `exp3_option_hedging.py` : deux couches (32, 32) en mode rapide ou (64, 64) en mode long; architecture spécialisée affine/quadratique en richesse.
- `exp4_gas_storage.py` : ClassifPI en (20, 20), Hybrid-Now en (24, 24) en mode rapide et (32, 32, 16) en mode long; softmax avec masque pour les actions discrètes.
- `exp5_microgrid.py` : ClassifHybrid en (32, 32), activation ELU, sortie mixte discrète-continue, contrôle continu borné.

11.2 Dépendances

Le fichier `requirements.txt` liste les dépendances Python principales :

```
numpy
scipy
pandas
matplotlib
torch
tqdm
jupyter
nbformat
```

11.3 Commandes utiles

Depuis la racine du package :

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python scripts/exp1_semilinear_pde.py --fast --out results
python scripts/exp2_linear_quadratic.py --fast --out results
python scripts/exp3_option_hedging.py --fast --out results
python scripts/exp4_gas_storage.py --fast --out results
python scripts/exp5_microgrid.py --fast --out results
python scripts/run_all.py --fast --out results
```

12 Conclusion

Le projet reproduit les idées centrales du papier : programmation dynamique en horizon fini, apprentissage de politiques par réseaux, régression de fonction valeur, quantification du bruit et benchmark Qknn en faible dimension. La conclusion principale est nuancée. Les résultats les plus solides sont Qknn pour le stockage de gaz et ClassifHybrid pour le micro-réseau. Les résultats LQ sont bons structurellement. La PDE est qualitativement correcte mais moins précise que le papier. La couverture d'option, en revanche, ne doit pas être présentée comme réussie quantitativement dans l'état actuel. Cette distinction est importante : elle montre que les algorithmes de deep learning pour le contrôle stochastique peuvent être puissants, mais nécessitent des choix d'entraînement, de mesure d'échantillonnage et d'architecture très soigneux.